

Prior-Auth Claims AI Pipeline

A Proof-of-Concept Redesign: 11 Stages, Rebuilt for a 5–7 Day Solo Build

The current system is, by its own audit, a single well-prompted Gemini call wearing a 12-service architecture costume. That's not a criticism of the POC instinct — it's actually the right instinct, just applied inconsistently. Some stages (ICD coding) already show the correct pattern: deterministic-first, AI-as-fallback, validated twice. Other stages (OCR, classification, evidence extraction) skipped that pattern entirely and handed everything to one ungrounded multimodal call.

This redesign applies the ICD module's winning pattern — **local/deterministic logic first, LLM only where it earns its cost, validation after every AI step** — to all 11 stages. Nothing here requires a real OCR engine, a vector database, Kubernetes, or a production database. Every change is chosen because it makes the live demo visibly more trustworthy: page-cited fields, rejected hallucinations shown on screen, a readiness score the audience can see move in real time.

At a Glance

Stage	Difficulty	Est. Time
1. Document Upload & Pre-flight Split	Easy	2–3 hrs
2. OCR (page-level, grounded)	Medium	0.5–1 day
3. Document Classification	Easy	2 hrs (folded into OCR)
4. Information Extraction (grounded)	Medium	3–4 hrs
5. ICD Coding	Easy	2–3 hrs (mostly reuse)
6. Policy / Cap Validation	Medium	3–4 hrs
7. Claim Readiness Score	Easy	2 hrs
8. TPA Query Prediction	Medium	4–5 hrs
9. Evidence Grounding (shared service)	Medium	3–4 hrs
10. Claim Packet Generation	Easy	3–4 hrs
11. Patient Registration & Persistence	Easy	2–3 hrs

Total estimated effort: roughly 4.5–5.5 focused developer-days for the core pipeline, leaving 1.5–2 days of buffer for wiring the UI, seeding demo data, and rehearsing the demo script — comfortably inside a 5–7 day window.

Stage-by-Stage Redesign

STAGE 1 Document Upload & Pre-flight Split	
CURRENT APPROACH	User selects a file, it's FileReader-base64'd, and the entire blob (even a 57-page PDF) is handed straight into the extraction call with no pre-processing.
POC APPROACH	Add a 10-line pre-flight step: split the PDF into pages with pypdf, generate a lightweight page thumbnail list, and tag each page with an index before anything touches an LLM. Cap demo PDFs at a sane page count (e.g. 20) and show a simple 'Pages detected: 14' chip in the UI.
WHY IT'S BETTER	This single change is the root fix for the 'page-offset error' and hallucination symptoms described in the audit — every downstream stage can now say <i>which page</i> a fact came from, which is the single most convincing thing to show a non-technical demo audience.
DIFFICULTY	Easy Est. time: 2–3 hrs

STAGE 2 OCR (page-level, grounded)	
CURRENT APPROACH	One Gemini multimodal call ingests the whole document as a single base64 blob and does OCR, classification, and extraction together with no page tracking and one global, self-reported confidence score.
POC APPROACH	Loop per page (using the split from Stage 1): call Gemini (or Tesseract for a free, fully offline fallback) once per page to get plain text, store it as <i>page_1_text</i> , <i>page_2_text</i> ... in a simple in-memory/JSON store. This raw per-page text becomes the 'ground truth' string that every later extraction claim must be checked against.
WHY IT'S BETTER	Turns OCR from an unverifiable black box into a small set of plain-text artifacts you can literally show on screen next to the fields extracted from them — this is what unlocks real grounding in Stage 4 and Stage 9, and it's the audit's #1 recommended fix, achievable without any real OCR engine.
DIFFICULTY	Medium Est. time: 0.5–1 day

STAGE 3 Document Classification

CURRENT APPROACH

Not a separate step — `document_type` is just one more field the single Gemini extraction call happens to emit, with nothing checking whether it's actually right.

POC APPROACH

Keep it as an LLM call, but make it its own tiny, cheap classification prompt (few-shot, 8 categories, temperature 0) run once per page-group *before* the extraction prompt, and log a confidence per document. If classification confidence is low, visibly flag the document as 'needs manual review' in the UI instead of silently extracting garbage fields.

WHY IT'S BETTER

Separating classification from extraction means a misclassified lab report doesn't quietly poison the extraction schema chosen for it — and a visible low-confidence flag is exactly the kind of 'the system knows what it doesn't know' moment that makes a healthcare demo credible.

DIFFICULTY

Easy

Est. time: 2 hrs (folded into OCR step)

STAGE 4 Information Extraction (grounded)

CURRENT APPROACH

A second giant Gemini call asks for structured clinical fields plus a 'verbatim source quote' per field — but nothing in code ever checks that the quote actually exists in the source document, so the grounding is enforced only by prompt wording.

POC APPROACH

Reuse the audit's own best idea, `isEvidenceCitationPlausible` (currently buried in the appeal generator) — promote it into a shared 20-line grounding check and run it immediately after every extraction call, against the page text from Stage 2. Any field whose quote doesn't substring-match (or hit ~40% token overlap) gets tagged 'unverified' in the UI instead of silently trusted.

WHY IT'S BETTER

This closes the exact gap the audit flags as 'the real gap' — and because it's copy-pasting logic that already exists and already works elsewhere in the codebase, it's a same-day win, not new R&D.;

DIFFICULTY

Medium

Est. time: 3–4 hrs

STAGE 5 ICD Coding

CURRENT APPROACH

Already the strongest module: Hinglish normalization → hardcoded edge-case rules → local JSON lookup → plausibility gate → AI fallback → re-validation → WHO-code truncation.

POC APPROACH

Keep the pipeline exactly as designed — it's already the deterministic-first pattern this whole redesign is copying elsewhere. Only two changes: move the ~12 hardcoded plausibility rules into a small JSON config (category → allowed ICD prefix) instead of if/else chains, and persist corrected codes to a local JSON/SQLite file instead of an in-memory array so the 'learning loop' survives a page refresh during the demo.

WHY IT'S BETTER

Proves to the demo audience that corrections made in the UI 'stick' — a small, cheap change that turns the weakest complaint about an otherwise-strong module ('resets on reload') into a non-issue, without touching the logic that already works.

DIFFICULTY

Easy

Est. time: 2–3 hrs (mostly reuse)

STAGE 6 Policy / Cap Validation

CURRENT APPROACH

Cost estimation and cap-checking are two disconnected code paths; cost estimation never checks the cap it computes room rent against, and unknown ICD codes fall back to one flat hardcoded price table with no per-specialty variance.

POC APPROACH

Merge into a single `estimateAndValidate(icdCode, admissionDetails)` function: compute the cost breakdown, immediately run it through the insurer cap table (config JSON, ~15 demo insurers), and return one combined object with {billed, allowed, deduction, reason} rather than two separate silent code paths.

WHY IT'S BETTER

A single function that outputs a real deduction breakdown (not just a vague warning) is far more demo-able — you can show a claims officer exactly '■7,000 billed → ■5,000 allowed → ■2,000 flagged for proportional deduction' in one screen instead of asking them to connect two disconnected outputs themselves.

DIFFICULTY

Medium

Est. time: 3–4 hrs

STAGE 7 Claim Readiness Score

CURRENT APPROACH

Pure arithmetic scoring, which is fine — but one input (room rent) is a hardcoded mock value (8000) instead of the real cost estimate, so the score is partly computed on fake data even though it looks fully dynamic in the UI.

POC APPROACH

Wire the score to the merged output from Stage 6 (real billed/allowed values) and keep the rest of the arithmetic model as-is — it doesn't need an LLM or ML, it's already the right level of complexity for a POC.

WHY IT'S BETTER

One-line-scale fix that removes a landmine the audit specifically flags as 'dead-code-quality, not wired to real data' — cheap to fix, embarrassing to demo if left broken, since anyone who changes room rent in the UI will notice the score doesn't move.

DIFFICULTY

Easy

Est. time: 2 hrs

STAGE 8 TPA Query Prediction

CURRENT APPROACH

Three deterministic rules plus a fourth Gemini call for anything the rules miss; deduplication between rule-based and AI-suggested queries is a crude 'first 10 characters match' check that misses paraphrases.

POC APPROACH

Keep the three deterministic rules and the AI fallback exactly as designed (this is another correctly-shaped module). Replace the substring dedup with a cheap embedding-similarity check using a small local sentence-embedding model (or even a TF-IDF cosine similarity — no GPU, no external service needed) so paraphrased duplicates actually get merged.

WHY IT'S BETTER

Small, contained fix to a named weakness that meaningfully improves what the demo audience sees — a query list with three near-identical questions looks unpolished, and this is a half-day fix, not an architecture change.

DIFFICULTY

Medium

Est. time: 4–5 hrs

STAGE 9 Evidence Grounding (shared service)

CURRENT APPROACH

Grounding logic exists but is inconsistent: rigorous at appeal-generation time (`isEvidenceCitationPlausible`), completely unverified at extraction time — two code paths with two different rigor levels for the same concept.

POC APPROACH

Extract the citation-plausibility check into one shared `evidenceGroundingService.ts` (as the audit itself recommends) and call it from both Stage 4 extraction and the appeal generator. No new logic — just make the one good implementation the single source of truth instead of writing it twice.

WHY IT'S BETTER

This is the highest-leverage, lowest-effort fix in the whole redesign: it doesn't add a new concept, it just stops duplicating an existing good one, and it directly fixes the audit's #3 flagged structural risk.

DIFFICULTY

Medium

Est. time: 3–4 hrs

STAGE
10

Claim Packet Generation

CURRENT
APPROACH

Template-based string assembly from the PreAuthRecord fields — already the right, non-LLM approach for a legally-formatted IRDAI form; only the medical-necessity narrative sentence uses an LLM call.

POC
APPROACH

Keep the template assembly unchanged. Add one visible feature: every field on the generated Part C/D form gets a small inline citation ('p.4') pulled from the Stage 1–4 page tracking, so the final document itself displays its own provenance.

WHY IT'S
BETTER

Turns the final deliverable into the most convincing artifact in the whole demo — a claims officer can literally click a field on the generated form and see the source page, which no part of the current system can do today.

DIFFICULTY

Easy

Est. time: 3–4 hrs

STAGE
11

Patient Registration & Persistence

CURRENT
APPROACH

masterPatientRecord.ts persists to whatever the current storage layer is (browser-side, not a real database); continuous-learning corrections are in-memory only and vanish on reload.

POC
APPROACH

Use a single local SQLite file (via better-sqlite3 or Python) as the entire persistence layer — one file, no server setup, no schema migrations tooling. Store patients, ICD corrections, and generated packets in three simple tables.

WHY IT'S
BETTER

SQLite gives real persistence across reloads — the single most common 'gotcha' moment in a live demo (refresh the page, everything's gone) — for near-zero setup cost and zero new infrastructure to explain or maintain.

DIFFICULTY

Easy

Est. time: 2–3 hrs

Suggested 5–7 Day Build Order

Ordered so that every day ends with something demo-able, not a half-finished stage.

Day 1	Stage 1 (page split) + Stage 2 (per-page OCR loop). Demo checkpoint: upload a multi-page PDF, see per-page text extracted with page numbers.
Day 2	Stage 3 (classification) + Stage 4 (extraction) + wire the grounding check stub. Demo checkpoint: extracted fields shown with source page + a 'verified/unverified' badge.
Day 3	Stage 9 (promote grounding into shared service, apply to both call sites) + Stage 5 cleanup (config-driven ICD rules, SQLite persistence). Demo checkpoint: correct an ICD code, refresh the page, correction survives.
Day 4	Stage 6 (merge cost estimation + cap validation) + Stage 7 (fix readiness score wiring). Demo checkpoint: change room rent in the UI and watch both the deduction and the readiness score move together.
Day 5	Stage 8 (embedding-based query dedup) + Stage 10 (citations on the generated Part C/D form). Demo checkpoint: generate a claim packet and click a field to see its source page.
Day 6	Stage 11 (SQLite persistence for patients/records) + end-to-end pass linking all stages through one orchestrator function. Demo checkpoint: full flow, upload to generated packet, works uninterrupted.
Day 7 (buffer)	Seed 3–4 realistic demo documents, rehearse the walkthrough, fix rough edges found while rehearsing. No new features.

What This Redesign Deliberately Leaves Out

- Real OCR engines (Tesseract/Texttract) — Gemini-per-page is accurate enough for a POC and avoids new infra.
- Vector databases / embedding search for evidence — a substring + token-overlap check is enough to demonstrate grounding.
- Kubernetes, Kafka, RabbitMQ, or any distributed processing — everything here runs synchronously, in-process, same as today.
- A production database — one SQLite file replaces the current ad-hoc browser persistence with zero new ops burden.
- A standalone document-classifier model — a cheap, separated LLM prompt is enough to remove the current entanglement risk.